

TESTE EVERTEC - Function CalcularDescontoCliente

1. Habilitei o Query Store para me ajudar na análise de performance da function enviada por vocês x a function atualizada por mim.

```
ALTER DATABASE [Teste_Evertec]
SET QUERY_STORE = ON;
```

100 %

Resultados Mensagens

	name	is_query_store_on
1	Teste_Evertec	1

2. Criação de Índices, PK e FK para melhoria na performance ao executar a Function.

[-]	[+]	dbo.Clientes
	[+]	Colunas
	[-]	Chaves
		PK_Clientes
	[+]	Restrições
	[+]	Gatilhos
	[-]	Índices
		PK_Clientes (Clusterizado)
	[+]	Estatísticas
[-]	[+]	dbo.Produtos
	[+]	Colunas
	[-]	Chaves
		PK_Produtos
	[+]	Restrições
	[+]	Gatilhos
	[-]	Índices
		PK_Produtos (Clusterizado)
	[+]	Estatísticas
[-]	[+]	dbo.Vendas
	[+]	Colunas
	[-]	Chaves
		PK_Vendas
		FK_Vendas_Clientes
	[+]	Restrições
	[+]	Gatilhos
	[-]	Índices
		IX_Vendas_Cliente_Status_DataVenda (Não Exclusivo, Não Clusterizado)
		PK_Vendas (Clusterizado)
	[+]	Estatísticas
[+]	[+]	dbo.VENDAS_TEMP
[-]	[+]	dbo.VendasItens
	[+]	Colunas
	[-]	Chaves
		PK_VendasItens
		FK_VendasItens_Produto
		FK_VendasItens_Vendas
	[+]	Restrições
	[+]	Gatilhos
	[-]	Índices
		IX_VendasItens_VendaID (Não Exclusivo, Não Clusterizado)
		PK_VendasItens (Clusterizado)

3. Remoção de todos ISNULL no WHERE, exemplo:

Antes:

```
SELECT
    @ClienteAtivo = CASE WHEN COUNT(*) > 0 THEN 1 ELSE 0 END
FROM Clientes C
WHERE C.ClienteID = @ClienteID
AND ISNULL(C.Status, '') <> 'Bloqueado';
```

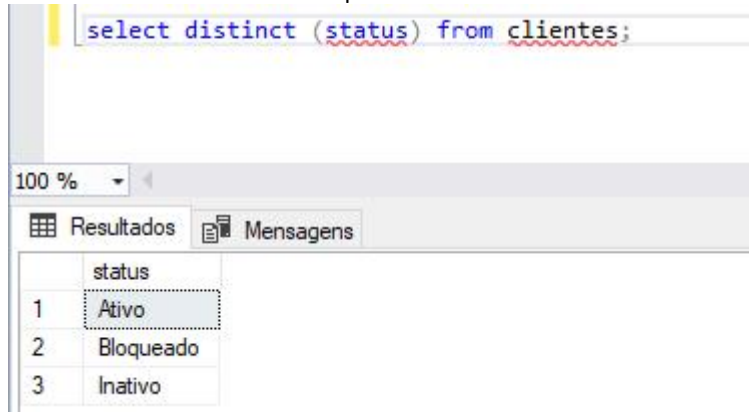
Depois:

```
SELECT
    @ClienteAtivo = CASE WHEN COUNT(*) > 0 THEN 1 ELSE 0 END
FROM Clientes C
WHERE C.ClienteID = @ClienteID
AND C.Status <> 'Bloqueado';
```

Explicação:

Utilizando a função ISNULL no filtro pode prejudicar a performance do índice, neste caso o campo deveria ser tratado diretamente na tabela, ou então incluir mais uma condição no WHERE de **AND C.STATUS IS NOT NULL**.

Na tabela de clientes existem apenas 3 status:



The screenshot shows a SQL query window with the following query:

```
select distinct (status) from clientes;
```

Below the query window, the 'Results' tab is selected, displaying the following data:

	status
1	Ativo
2	Bloqueado
3	Inativo

4. O uso do FORMAT deixa o SQL lento, pois ele impede o uso eficiente do índice e também um alto processamento.

Antes:

```
SELECT
    @QtdComprasCliente = COUNT(DISTINCT V.VendaID)
FROM Vendas V
WHERE V.ClienteID = @ClienteID
AND V.Status = 'Concluida'
AND FORMAT(V.DataVenda, 'yyyy-MM-dd') <= FORMAT(@DataAnalise, 'yyyy-MM-dd');
```

Depois:

```
SELECT
    @QtdComprasCliente = COUNT(DISTINCT V.VendaID)
FROM Vendas V
WHERE V.ClienteID = @ClienteID
AND V.Status = 'Concluida'
AND V.DataVenda < DATEADD(DAY, 1, CAST(@DataAnalise AS DATE));
```

Explicação:

- uso do CAST para retornar somente o dia, sem as horas do campo DataAnalise;
- uso DATEADD(DAY,1 soma um dia por questão das horas de um dia X, para buscar tudo o que tiver entre 00:00:00 e 23:59:59

5. Melhoria na consulta para variável @ClienteAtivo

Antes:

```
SELECT
    @ClienteAtivo = CASE WHEN COUNT(*) > 0 THEN 1 ELSE 0 END
FROM Clientes C
WHERE C.ClienteID = @ClienteID
AND C.Status <> 'Bloqueado';
```

Depois:

```
SELECT
    @ClienteAtivo = CASE WHEN EXISTS (SELECT 1 FROM Clientes C WHERE C.ClienteID = @ClienteID AND C.Status <> 'Bloqueado')
    THEN 1 ELSE 0 END;
```

Explicação:

Este SELECT tem apenas a função de identificar se existe um cliente ativo e que não seja bloqueado, neste caso o ideal é utilizar a função EXISTS que é mais eficiente ao invés do COUNT(*).

6. Junção de SELECTS da tabela vendas.

Antes:

```
SELECT
    @QtdComprasCliente = COUNT(DISTINCT V.VendaID)
FROM Vendas V
WHERE V.ClienteID = @ClienteID
AND ISNULL(V.Status, '') = 'Concluida'
AND FORMAT(V.DataVenda, 'yyyy-MM-dd') <= FORMAT(@DataAnalise, 'yyyy-MM-dd');

SELECT
    @TotalComprado = SUM(ISNULL(VI.Quantidade, 0) * ISNULL(VI.ValorUnitario, 0))
FROM Vendas V
INNER JOIN VendasItens VI
    ON V.VendaID = VI.VendaID
WHERE V.ClienteID = @ClienteID
AND ISNULL(V.Status, '') = 'Concluida'
AND V.DataVenda < DATEADD(DAY, 1, CAST(@DataAnalise AS DATE));

IF ISNULL(@TotalComprado, 0) > 10000
    SET @Desconto += 10;
ELSE IF ISNULL(@TotalComprado, 0) BETWEEN 5000 AND 10000
    SET @Desconto += 5;

SELECT
    @QtdCategorias = COUNT(DISTINCT P.CategoriaID)
FROM Vendas V
INNER JOIN VendasItens VI
    ON V.VendaID = VI.VendaID
INNER JOIN Produtos P
    ON VI.ProdutoID = P.ProdutoID
WHERE V.ClienteID = @ClienteID
AND ISNULL(V.Status, '') = 'Concluida'
AND V.DataVenda < DATEADD(DAY, 1, CAST(@DataAnalise AS DATE));
```

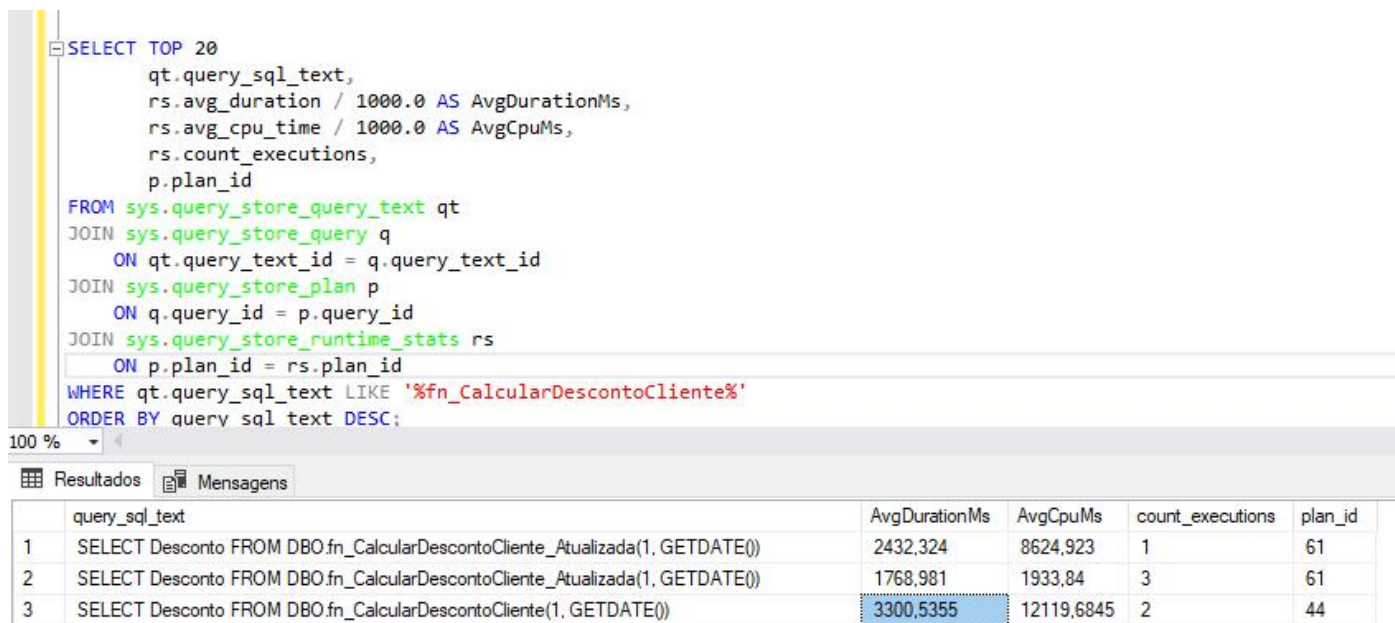
Depois:

```
SELECT
    @QtdComprasCliente = COUNT(DISTINCT V.VendaID),
    @TotalComprado = SUM(ISNULL(VI.Quantidade, 0) * ISNULL(VI.ValorUnitario, 0)),
    @QtdCategorias = COUNT(DISTINCT P.CategoriaID)
FROM Vendas V
INNER JOIN VendasItens VI ON (V.VendaID = VI.VendaID)
INNER JOIN Produtos P ON (VI.ProdutoID = P.ProdutoID)
WHERE V.ClienteID = @ClienteID
AND V.Status = 'Concluida'
AND V.DataVenda < DATEADD(DAY, 1, CAST(@DataAnalise AS DATE));
```

Explicação:

Ambas queries realizam consulta na **tabela de vendas** com o status de **concluída**, neste caso deixei apenas uma query, inserindo as variáveis **@QtdComprasCliente** e **@TotalComprado** em um único SELECT.

7. Realizei uma análise de desempenho utilizando o Query Store e usei o Gemini para trabalharmos com percentual de redução, e o resultado foi o seguinte



```
SELECT TOP 20
    qt.query_sql_text,
    rs.avg_duration / 1000.0 AS AvgDurationMs,
    rs.avg_cpu_time / 1000.0 AS AvgCpuMs,
    rs.count_executions,
    p.plan_id
FROM sys.query_store_query_text qt
JOIN sys.query_store_query q
    ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p
    ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs
    ON p.plan_id = rs.plan_id
WHERE qt.query_sql_text LIKE '%fn_CalcularDescontoCliente%'
ORDER BY query_sql_text DESC;
```

	query_sql_text	AvgDurationMs	AvgCpuMs	count_executions	plan_id
1	SELECT Desconto FROM DBO.fn_CalcularDescontoCliente_Atualizada(1, GETDATE())	2432,324	8624,923	1	61
2	SELECT Desconto FROM DBO.fn_CalcularDescontoCliente_Atualizada(1, GETDATE())	1768,981	1933,84	3	61
3	SELECT Desconto FROM DBO.fn_CalcularDescontoCliente(1, GETDATE())	3300,5355	12119,6845	2	44

- redução de 84% no uso da CPU
- melhoria de 46% na velocidade de execução

Provando eficácia no desempenho da função através dos ajustes realizados acima.

Questões do Bloco E

8. Quais recursos e ferramentas utilizou na análise do problema de lentidão e como as utilizou.

Habilitei e utilizei o Query Store, ferramenta própria do MSSQL para realizar análise de tempo e consumo de CPU, através da query mencionada no item 7. E também o uso do Gemini para fazer um levantamento deste resultado com base em percentuais, que nos dão uma visão melhor sobre as melhorias.

9. Quais problemas de arquitetura e implementação foram identificados na função fornecida.

Falta de Chaves Primárias, Chaves Estrangeiras e Índices

Uso de funções que causam lentidão e não são recomendadas quando a coluna possui índice, como o FORMAT e ISNULL no WHERE.

10. Pontos de gargalo.

Com base na minha análise e nos ajustes que realizei foram as funções dentro do WHERE, como:

WHERE ISNULL

WHERE FORMAT

O uso do COUNT(*) dentro da tabela de clientes apenas para validar clientes ativos.

Porém existem outros pontos de gargalo que gostaria de tratar se o tempo do teste de estende-se por mais um dia.

11. Quais medidas tomou para solucionar lentidão.

- Junção de queries que possuíam a mesma condição, realizando em uma única query a junção das variáveis.
- Criação de PK, FK e Index.
- Remoção do FORMAT em campos de data, utilizando o CAST e DATEADD.
- Otimização do SELECT que retorna o cliente ativo, utilizando EXISTS ao invés do COUNT(*)
- Remoção da função ISNULL dentro dos filtros (WHERE), pois esta função impacta diretamente no tempo de execução e se há necessidade de alteração a mesma deve ser feita diretamente na tabela.

12. Por que tomou tais medidas e como elas impactam o desempenho da query atual.

Todas as medidas que foram adotadas já eram de meu conhecimento ao longo dos anos trabalhando com SQL e MySQL. No item 7 eu explico detalhadamente como a nova function desempenhou um papel com ganho de performance.

13. Descrever quais alterações trouxeram maior ganho de desempenho e porquê.

Com certeza a criação das chaves primárias, estrangeiras e índices trazem melhorias muito boas, afim de criar o relacionamento entre as tabelas utilizadas e com os índices ajuda o SQL a não fazer scans completos nas tabelas.

E também a remoção do ISNULL no WHERE, onde não há necessidade nenhum de tratar os campos no WHERE, essa função deve ser executada e ajustada, se necessário, diretamente na tabela.

Questões do Bloco F

14. Você usou IA neste teste? Se sim, descreva 1 trecho em que ajudou e quais validações você fez antes de aceitar a resposta.

Utilizei o Gemini para me trazer percentual de melhoria entre as functions, mas sempre com base nos dados apresentados no Query Store, como o tempo de processamento e consumo de CPU, descritos no item 7.

15. Cite um risco de usar IA em SQL Server e como você mitigaria.

Acredito que o maior risco é justamente você achar que a IA irá resolver uma questão como atualização deste function sozinha, e após qualquer alteração, por exemplo no WHERE ISNULL eu realizei a verificação na tabela de clientes para identificar se havia algum status como NULO, antes de remover a função. Ou seja, acredito que o maior risco que se pode ocorrer é quando realizamos ajustes na query e ela passa a retornar um valor diferente, sem que haja uma validação antes.